

RESEARCH

Open Access



Dependency-aware online task offloading based on deep reinforcement learning for IoV

Chunhong Liu^{1,2}, Huaichen Wang¹, Mengdi Zhao¹, Jialei Liu^{3*}, Xiaoyan Zhao^{1,2} and Peiyan Yuan^{1,2*}

Abstract

The convergence of artificial intelligence and in-vehicle wireless communication technologies, promises to fulfill the pressing communication needs of the Internet of Vehicles (IoV) while promoting the development of vehicle applications. However, making real-time dependency-aware task offloading decisions is difficult due to the high mobility of vehicles and the dynamic nature of the network environment. This leads to additional application computation time and energy consumption, increasing the risk of offloading failures for computationally intensive and latency-sensitive applications. In this paper, an offloading strategy for vehicle applications that jointly considers latency and energy consumption in the base station cooperative computing model is proposed. Firstly, we establish a collaborative offloading model involving multiple vehicles, multiple base stations, and multiple edge servers. Transferring vehicular applications to the application queue of edge servers and prioritizing them based on their completion deadlines. Secondly, each vehicular application is modeled as a directed acyclic graph (DAG) task with data dependency relationships. Subsequently, we propose a task offloading method based on task dependency awareness in deep reinforcement learning (DAG-DQN). Tasks are assigned to edge servers at different base stations, and edge servers collaborate to process tasks, minimizing vehicle application completion time and reducing edge server energy consumption. Finally, simulation results show that compared with the heuristic method, our proposed DAG-DQN method reduces task completion time by 16%, reduces system energy consumption by 19%, and improves decision-making efficiency by 70%.

Keywords Internet of Vehicles, Task offloading, Dependency awareness, Deep reinforcement learning

Introduction

5G mobile communication technology enables interconnection of all things through the establishment of a more efficient, intelligent, and reliable heterogeneous communication network, achieving ultra-low latency and

high-speed data transmission [1–5]. With the support of intelligent network technology based on AI, edge computing, and the Internet of Things [6], IoV technology deeply integrates cars with the network. It enables intelligent interconnection among vehicles and between vehicles and road networks [7–9]. This brings users a more intelligent, convenient, and secure travel experience. A large number of applications in vehicular environments appear with the rapid growth in the number of intelligent vehicles. Novel vehicle applications continue to evolve, such as driver assistance, autonomous driving, collision warning and high-precision mapping. This development not only brings new business models and growth opportunities to the automotive industry, but also enhances convenience and enjoyment in people's travel and daily lives [10, 11]. At the same time, novel vehicle applications

*Correspondence:

Jialei Liu
jjialeiliu@hbuas.edu.cn
Peiyan Yuan
peiyuan@htu.edu.cn

¹ College of Computer and Information Engineering, Henan Normal University, Xinxang 453007, China

² Smart Business And Internet of Things Technology, Henan Provincial Engineering Laboratory, Xinxang 453007, China

³ Computer School, and Hubei Key Laboratory of Power System Design and Test for Electrical Vehicle, Hubei University of Arts and Science, Xiangyang 441053, China



© The Author(s) 2024. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

impose higher requirements on the IoV system, including computational capacity, processing latency, and energy consumption. More computational resources are necessary to ensure the high reliability and efficiency of data processing [12].

Edge data increases in IoV and its limited capacity cannot process all the data in time. cloud computing architecture has high latency and low reliability due to Long-distance transmission. Meeting quality of service requirements for real-time vehicular applications is a challenge [13]. Edge computing provides cloud services at the edge through wireless access networks. This extends cloud computing capabilities closer to the user side and significantly reduces the transmission latency of user service requests [14, 15]. Vehicle edge computing is a new computing paradigm and is gaining attention in the field of intelligent transportation systems in recent years [16]. The way vehicle applications are offloaded from resource-constrained vehicles to lightweight, ubiquitous edge servers can lead to enhanced vehicle performance. Application offloading decision-making is primarily about how to offload vehicle applications and where to offload them. The main optimization objectives of offloading decisions include latency, energy consumption, and trade-offs between latency and energy consumption [17].

However, the rapid expansion of intelligent vehicles has led to the emergence of a plethora of vehicle applications in the IoV. Edge servers are at risk of overload in computational storage and bandwidth resources. Offloading all vehicle applications to a single edge server may cause overload and limit performance gains. When performing edge collaborative offloading of vehicle applications, it is essential to consider the impact of task dependencies on application latency [18]. For example, there are certain dependencies between various components in an augmented vehicle augmented reality system. Such as target tracking, model mapping, target recognition, perspective transformation and fusion processing. Considering task dependencies in task offloading method ensures that multiple vehicle applications are completed promptly.

Due to high mobility of vehicles, it is crucial for the system. Making vehicle applications timely offloading decisions and being completed within extremely low latency. Otherwise, there is a risk of offloading failure for vehicle applications. Additionally, making offloading decisions is difficult because of the dynamically changing vehicular network environment and the rapidly changing wireless channel conditions.

In this paper, we propose a Dependency-aware Online Task Offloading method to address above problems. This method take interdependencies among multiple tasks into consideration. Vehicle make applications consisting of tasks with multiple dependencies offloading to edge

servers of base stations. Edge servers deploy dependency-aware online task offloading methods to autonomously make offloading decisions in dynamic and uncertain IoV. Edge server decides whether to transfer a dependency task to other edge servers or the current edge server for execution. By leveraging the computational resources of nearby edge servers to promptly process and fulfill emergency information or services for vehicles, we can enhance the quality of service for vehicles and reduce the energy consumption of IoV systems.

The main contributions of this study are as follows:

1. We establish a model for edge collaborative offloading. The model consists of multiple vehicles, multiple base stations (BS), and multiple edge servers (ES). The model divides complex applications into multiple tasks and uses a DAG to model the data dependencies among tasks.
2. A dependency-aware online DAG task offloading scheme is proposed, which solves data dependencies present in task offloading. The scheme defines application and task priorities and models data dependencies between tasks and tasks. We formulate offloading problem as a Markov Decision Process (MDP). It treats DAG task as part of the environment state to obtain a suitable offloading decision.
3. We experimented extensively to prove the effectiveness of reducing completion time of multiple applications and total energy consumption of edge servers, and the proposed DAG-DQN was superior in improving completion rate of applications and the efficiency of system decision making.

The remainder of this paper is organized as follows: [Related work](#) section summarizes related work; [System architecture](#) section describes the system architecture and model in detail; [Dependency-aware task offloading method](#) in [Dependency-aware task offloading method](#) section. [Experimental evaluation](#) section presents the experimental evaluation. Finally, [Conclusion and future work](#) section concludes this paper and discusses future work.

Related work

This section discusses the current state of task offloading on edge computing research. Then, works on latency, energy consumption, and task dependency are presented, and related works on task offloading are compared in Table 1.

Task latency is the main optimization objective for executing task offloading in IoV [29]. Fan et al. [19] proposed a collaborative resource management scheme to minimize the processing latency of vehicle tasks. This

Table 1 Compare the characteristics of the task offload work

Categories	Scenes		DAG	Define priorities		DRL	Decision Objectives	
	IoV	Other		Application	Task		Finish time	Energy consumption
[19]	✓	×	×	×	×	×	✓	×
[20]	×	✓	×	×	×	✓	✓	×
[21]	×	✓	×	×	×	✓	✓	×
[22]	✓	×	×	×	×	×	✓	✓
[23]	✓	×	×	×	×	×	✓	✓
[24]	✓	×	×	×	×	✓	✓	✓
[25]	✓	×	✓	×	×	✓	✓	×
[26]	✓	×	✓	✓	✓	×	✓	×
[27]	✓	×	✓	×	✓	✓	✓	×
[28]	×	✓	✓	×	✓	×	✓	×
This paper	✓	×	✓	✓	✓	✓	✓	✓

scheme implements task offloading, channel assignment, and allocation of computational resources to vehicles and roadside devices. Zhou et al. [20] devised a sophisticated risk-sensitive reinforcement learning algorithm for online task offloading decisions, aimed at reducing the latency of computational tasks. Deng et al. [21] introduced an autonomous partial offloading system for latency-sensitive computational tasks, delivering users with minimal latency offloading services. Although these schemes effectively reduce task latency, they ignore the fact that edge devices consume large amounts of energy while performing highly complex computational tasks.

Reducing energy consumption is critical to improving IoV system performance, and it is another key optimization objective for offloading decisions. Energy consumption typically encompasses computational and communication energy costs. Reducing computational and communication energy costs of edge servers has significant implications for offloading decisions [30]. Many schemes jointly consider latency and energy consumption as optimization objectives for offloading decisions. Shinde et al. [22] proposed the utilization of a value iteration algorithm to seek the optimal strategy in uncertain vehicular environments, jointly optimizing total latency and energy consumption during vehicle task processing. Ning et al. [23] proposed an efficient and energy-saving task offloading framework that supports mobile edge computing, jointly considering task offloading between servers and the downlink energy consumption of roadside units. This framework aims to minimize the energy consumption of roadside units under task latency constraints. Maleki et al. [24] Consider energy consumption, transmission delay and processing delay during offloading as offloading costs. A switchable offloading strategy

is proposed to improve service quality and user experience. Zhao et al. [25] proposed a task offloading method based on deep Q-networks to achieve an optimal balance between task execution latency, processing speed and energy consumption.

While these works consider reducing task latency and energy consumption of edge devices, they do not consider the impact of task dependencies on offloading decisions. They treat applications as independent tasks and do not divide applications into multiple tasks. Complex applications can be divided into multiple tasks that can be executed independently on different edge servers [31]. Dependencies between tasks determine the execution order of tasks. Therefore, offloading methods should consider task dependencies. Liu et al. [26] considered task dependency requirements between multiple tasks of the same application and application completion time constraints to prioritize multiple applications and tasks. Define task offloading problem as an NP-hard optimization problem. Qi et al. [27] proposed a knowledge-driven decision framework for service offloading. Complex services are considered as multiple data-dependent tasks to reduce task response latency. He et al. [28] propose a task replication-based DAG task offloading method that uses DAG to model an application with a set of distributed tasks. Assigning tasks to different computing devices ensures that DAG tasks can be accomplished with lower latency.

In conclusion, these works do not consider the impact of data dependency requirements between tasks on offloading decisions. As well as the impact of task dependencies on application completion time and edge device energy consumption. Therefore, they cannot be directly applied to vehicular application offloading methods in

base station cooperative computing. In addition, prioritization must be performed before application processing, with fine-grained prioritization of tasks with data dependencies. Application completion time and energy consumption are optimized using collaborative offloading between edge servers.

To improve the ability to solve the dependent task offloading problem in scenarios involving multiple vehicles, base stations, and edge servers. In this paper, we propose a vehicle application offloading method for base station cooperative computing scenarios that integrates application completion time and energy consumption. The dependencies between tasks are represented as directed acyclic graphs. A deep reinforcement learning algorithm that incorporates DAG task coding is used to enable the agent to perceive task dependencies. This method solves the dependent task offloading problem and reduces the risk of vehicle application offloading failure.

System architecture

The system architecture is introduced first. Subsequently, the relevant models and problem formulations are defined. Details of the symbols used in the paper are shown in Table 2 for reference.

System architecture

Vehicles in IoV can use different types of resources and can offload tasks to edge servers for fast processing. However, in the event of a sharp increase in workload, the task completion time will also escalate. If additional edge servers are hastily deployed at the base station, improper configuration in the short term will require additional human and material resources, resulting in

wasted computing resources. Therefore, we propose a task offloading strategy in the inter-base station cooperative computing model to assist edge servers in managing the tasks offloaded by vehicles.

Figure 1 illustrates the system architecture of vehicular edge computing, composed of vehicles, BSs, and ESs. Suppose a crossroad is covered by K BSs located along the road, each with a fixed communication range. Each BS is equipped with an ES of varying performance and limited computational resources. The BSs located on either side of the road can communicate with each other, collectively forming an undirected topological structure.

Suppose M vehicles located at different positions are traveling at a constant speed v . Each requesting vehicle has a computationally intensive and latency-sensitive application that needs to be offloaded within strict deadlines. Each requesting vehicle performs offloading only once. Different vehicles have different applications to offload, and these applications are independent of each other. Offloading and execution do not interfere with each other. The requesting vehicle offloads its application program to the ES for execution via the BS. The ES receives the application program data from the requesting vehicle, performs the appropriate computational tasks, and returns the results to the vehicle. The application can be denoted as $APP_m = \{D_m^{up}, D_m^{down}, T_m^{max}\}$, $m \in M$ represents the amount of data to be uploaded to the ES for offloading application m , D_m^{down} represents the amount of data to be returned to the requesting vehicle after processing application m by the ES, and T_m^{max} represents the maximum deadline allowed for completing application m . If the application exceeds the maximum deadline then it faces processing failure. When the application fails it becomes worthless and hence there is a need to ensure that it completes on the edge node within the given deadline.

Within the system architecture, each base station is equipped with K sets of transceiver antennas, utilizing $K \times K$ channels for communication. The communication process is regarded as a multiple input multiple output (MIMO) system [32]. The transmission rate S of the MIMO architecture is:

$$S = B \log_2 \left[\det \left(I_K + \frac{SNR}{K} X \cdot X' \right) \right] \quad (1)$$

$$s.t. \sum_{i=1, j=1}^K |x_{ij}|^2 = K$$

where K denotes the number of transmit and receive antennas. B denotes the channel bandwidth. SNR denotes the average signal-to-noise ratio. I_K denotes the $K \times K$

Table 2 Summary of notations

Symbols	Description
K	Number of edge servers and base stations
M	Number of applications
N	Task collection
T_m^{up}	Transfer time of application m offloaded to BS
T_{ij}^{trans}	Transmission time between task i and task j
T^{app}	Application processing time
T_m^{down}	The time it takes application m to return results
T^{total}	Application completion time
f_k	ES computing power allocated to each task
$p_k^{trans}, p_k^{cal}, p_k^{static}$	ES transmission, computation and idle power
$E_{k,j}^{trans}$	ES transmission energy consumption
E_m	Total energy consumption spent to complete application m
α, β	Weighting factors

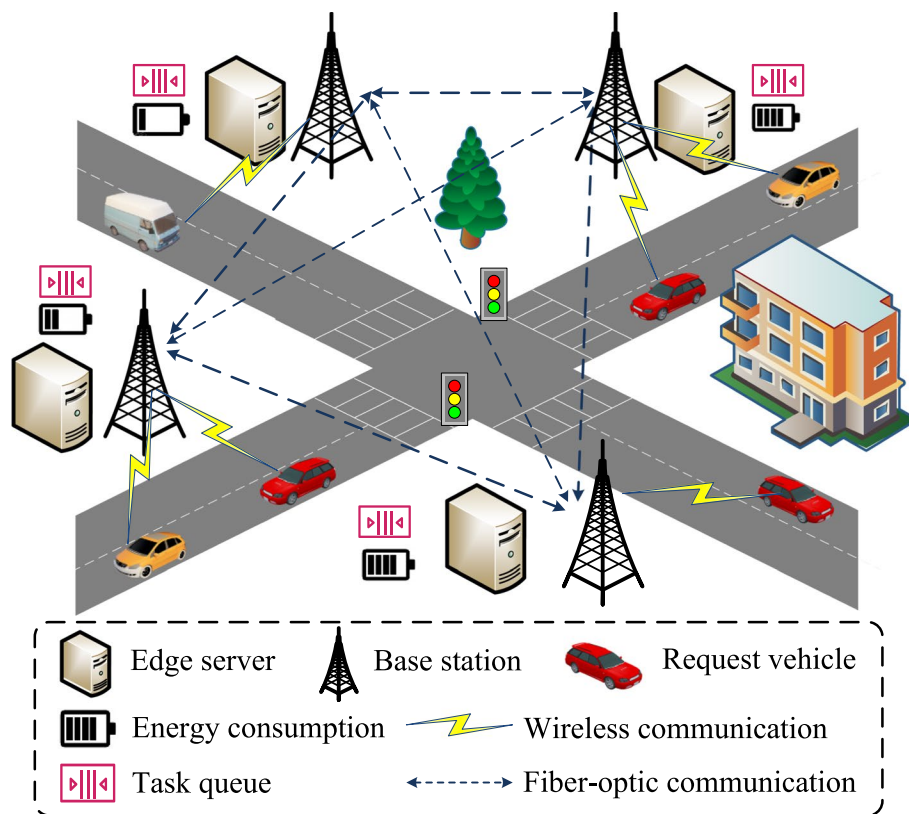


Fig. 1 System architecture

identity matrix. X represents a normalized channel matrix composed of x_{ij} ($1 \leq x_{ij} \leq K$), with its conjugate transpose denoted by X^* [33].

Task dependency model

Each vehicle application can be divided into multiple tasks, which are interconnected by data dependencies.

Some tasks require the output data from other tasks as input after their completion. As illustrated in Fig. 2, for example, the real-time navigation application in vehicles can visually represent the data dependencies among tasks, where data is transferred between different tasks to achieve real-time navigation functionality. Considering tasks as computational units, each task can be

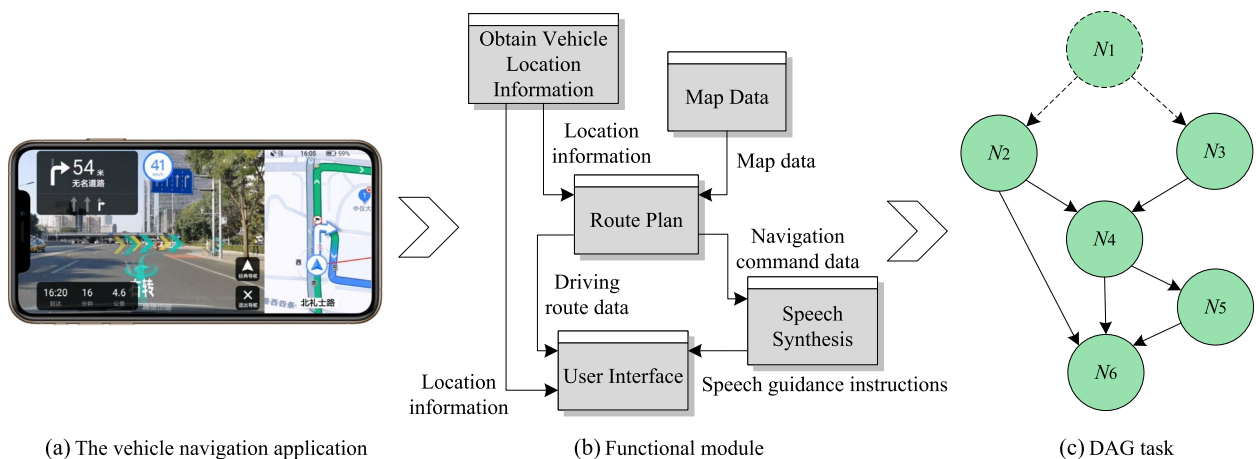


Fig. 2 Model the vehicle application as a DAG task

offloaded for execution on ES. Modeling the application as a directed acyclic graph, denoted as $G = \langle N, E \rangle$, where N represents the set of tasks, with n denoting the number of tasks in the set, $N = [N_1, N_2, \dots, N_n]$, and $E_{i,j} \in E$ representing the set of data edges. A data edge $E_{i,j}$ indicates that the execution of task N_j depends on the results of task N_i . In the directed acyclic graph, nodes represent tasks, where the value on node i signifies the amount of data processing required to complete task N_i , denoted as c_i . The edges between nodes represent data dependencies between two tasks, where the weight of the edge signifies the amount of data transmitted between tasks with dependencies, denoted as $d_{i,j}$ representing the data size transferred from task i to task j .

Figure 2 depicts the process of modeling the vehicle navigation application as a DAG of tasks. To maintain uniformity in the structure of the DAG task model, if an application has multiple initial or terminal tasks, a virtual task is defined as a new initial or terminal task. In Fig. 2b, the “Obtain Vehicle Location Information” and “Map Data” modules correspond to tasks N_2 and N_3 in Fig. 2c. Since these two tasks only involve data transmission without receiving data, a virtual task N_1 is defined to connect tasks N_2 and N_3 . The virtual task N_1 does not require any data processing or transmission. In this scenario, the virtual task N_1 serves as the new initial task, while N_6 acts as the terminal task. Tasks N_2 and N_3 are predecessors of task N_4 , with task N_4 depending on the execution results of N_2 and N_3 . Task N_4 is a predecessor of tasks N_5 and N_6 , and task N_5 is a predecessor of task N_6 . Task N_5 is a successor of task N_4 . Each task must wait for its predecessors to complete before it can commence execution. Tasks connected in series cannot be executed in parallel.

Application completion time

The completion time of each application consists of three parts. (1) the wireless transmission time to upload the application to BS; (2) the processing time of the application on ES; and (3) the wireless transmission time for the BS to send back the processed results of the application to the requesting vehicle. When application m is offloaded from the requesting vehicle to the nearby BS, the wireless transmission time is:

$$T_m^{up} = \frac{D_m^{up}}{S} \quad (2)$$

The processing time of an application consists of three parts: task computation time, task-to-task data transmission time, and task waiting time. The computation time of task i is:

$$T_i^{cal} = \frac{c_i}{f_k} \quad (3)$$

where f_k represents the computational capability allocated to task by the ES k .

For two tasks on the same ES, there is no need to consider data transmission time. However, tasks located on different ESs require data transmission. Define $\delta_{i,j}$ to indicate whether tasks i and j under the same application are offloaded to the same ES.

$$\delta_{i,j} = \begin{cases} 0, & \text{on the same ES} \\ 1, & \text{otherwise} \end{cases} \quad (4)$$

When the amount of data transmitted from the current task i to the subsequent task j is $d_{i,j}$, the transmission time is:

$$T_{i,j}^{trans} = \delta_{i,j} \frac{d_{i,j}}{S} \quad (5)$$

Each ES is capable of maintaining a task queue, where tasks awaiting processing are stored. The ES places tasks that need to be processed into the task queue. Considering that the ES can only compute one task at a time and follows the principle of First-Come-First-Serve in processing tasks in the queue. The waiting time for task i is defined as T_i^{wait} .

Let t_i^{start} and t_i^{end} denote the actual start time and actual end time of task i , while $pre(i)$ and $sub(i)$ represent the set of predecessor tasks and successor tasks of task i respectively. If a task has no predecessor tasks, it is referred to as a starting task and denoted as N^{in} ; if a task has no successor tasks, it is considered as an ending task and denoted as N^{out} . For the starting task N^{in} , it holds that $t_{N^{in}}^{start} = 0$.

In conclusion, the actual start time of any task will not precede the completion time of any of its predecessor tasks. Therefore, the start time t_i^{start} and end time t_i^{end} of task i are as follows:

$$t_i^{start} = \max \left\{ T_i^{wait}, \max_{j \in pre(i)} (t_j^{end} + T_{j,i}^{trans}) \right\} \quad (6)$$

$$t_i^{end} = t_i^{start} + T_i^{cal} \quad (7)$$

The processing time of the entire application is:

$$T^{app} = t_{N^{out}}^{end} - t_{N^{in}}^{start} \quad (8)$$

After the ES completes processing application m , BS returns the processed data results to the requesting vehicle. The transmission time required for returning the results is:

$$T_m^{down} = \frac{D_m^{down}}{S} \quad (9)$$

Finally, by combining Eqs. (2), (8), and (9), the completion time of the application is:

$$T^{total} = T_i^{up} + T_i^{app} + T_m^{down} \quad (10)$$

Energy consumption model

To evaluate the energy consumption of ES during task processing, three types of power are defined. (1) Computing power: the power consumed by the ES when performing computational tasks; (2) Transmission power: the power consumed by the ES during data transmission; (3) Standby power: the power consumed by the ES when in an active state but not processing tasks or transmitting data.

When ES k processes task i and needs to communicate with multiple ESs where the subsequent tasks of task i are located, the data transmission during this communication process is parallel. ES k simultaneously transmits the data generated by task i to multiple ESs unloading the subsequent tasks. The energy consumption during data transmission by ES k is:

$$E_{k,i}^{trans} = \max_{j^* \in sub(i)} T_{i,j^*}^{trans} \cdot P_k^{trans} \quad (11)$$

When the ES task queue is empty and not transmitting data to other ES, the ES is determined to be in an idle state. The idle time of ES k is:

$$T_k^{static} = T^{total} - \sum_{i \in \Phi(k)} \left(T_i^{cal} + \max_{j^* \in sub(i)} T_{i,j^*}^{trans} \right) \quad (12)$$

The symbol $\Phi(k)$ denotes the set of pending tasks in the task queue of ES k .

By combining Eqs. (11) and (12), the total energy consumption of all ES in the system when handling application m is:

$$E_m = \sum_{k=1}^K \left(T_k^{static} P_k^{static} + \sum_{i \in \Phi(k)} T_i^{cal} P_k^{cal} + \sum_{i \in \Phi(k)} E_{k,i}^{trans} \right) \quad (13)$$

Problem formulation

To assess whether an application can be completed before the deadline or before the vehicle exits the communication range. This paper defines the completion rate of application offloading to evaluate the performance of DAG-DQN method. The distance between vehicle m and the point where vehicle m exits the signal coverage of the

base station is denoted by $\mathcal{L}_k^m$. The completion time of application m must satisfy:

$$T_m^{total} \leq \min \left(T_m^{\max}, \frac{\mathcal{L}_k^m}{v} \right) \quad (14)$$

Equation (14) indicates that application m has been successfully processed within the deadline or before the requesting vehicle exits the communication range, and the requesting vehicle has received the processing results of application m . Meeting these conditions constitutes successful processing, denoted by $\chi_m = 1$. If these conditions are not met, it is considered a processing failure, denoted by $\chi_m = 0$. The completion rate of all applications offloaded to the base station at the same time is:

$$\mathbb{R} = \frac{\sum_{i=1}^M \chi_m}{M} \quad (15)$$

The optimization objective is defined as simultaneously minimizing the completion time and total energy consumption of all applications in the system. The completion rate of the applications is used as a constraint on the optimization objective. The optimization problem is formulated as follows:

$$\min OT = \left[\alpha \sum_{m=1}^M T_m + \beta \sum_{m=1}^M E_m \right] \quad (16)$$

Where α and β are weighting factors representing the importance of total application completion time and total energy consumption respectively. In Internet of Vehicles, there is a high demand for low-latency offloading of tasks, with the primary goal being to reduce total application completion time rather than decrease total energy consumption. Therefore, more weight should be assigned to $\sum_{i=1}^M T_i$ than to $\sum_{i=1}^M E_i$, implying that α should be greater than β .

The above optimization problem belongs to the assignment problem in 0-1 Integer Linear Programming, which is NP-hard and cannot be solved to optimality in polynomial time. Next, we will design a method to efficiently solve this problem and obtain a suboptimal solution.

Dependency-aware task offloading method

In this section, we design a task offloading method that jointly optimizes latency and energy consumption based on Deep Reinforcement Learning for Task Dependent Quotient Offloading, or DAG-DQN for short, to provide a suboptimal solution with low computational complexity. The task offloading process is as follows.

Multiple vehicles offloading multiple applications to ES at the same time, ES divides the application into DAG tasks in order according to the priority of the application. Multiple tasks under the same application are stored in the task queue according to the priority of the tasks. DAG-DQN is a reinforcement learning method based on task dependency awareness. It aims to jointly optimize latency and energy consumption. ESs deploying the DAG-DQN approach collaborate with other ESs to process the task of vehicle.

The DAG-DQN method is responsible for making the best decision, which relies on the agent's exploratory learning of task offloading rules from interaction with the environment. As shown in Fig. 3. Agent learns optimal task offloading rules and patterns by perceiving task dependencies. Agent perceives the task offloading environment and task state, makes offloading actions to interact with the environment in real time, and uses Q network as a strategy judgment criterion. States, actions, rewards and next states are stored in the experience replay pool. The Q network is trained repeatedly using the Q learning algorithm over many iterations and the network parameters are constantly updated to approximate the optimal Q function. The trained DAG-DQN method is evaluated in a simulated environment, and the evaluation metrics include application completion time, edge server energy consumption, and application

completion rate. Based on the evaluation results, we decide whether to deploy the DAG-DQN method to the real environment.

Prioritize building applications and tasks

By prioritizing applications and tasks, the system can better manage and schedule tasks to ensure that requesting vehicles are processed promptly within application completion deadlines and to avoid situations where tasks cannot be completed due to communication delays or base station communication range limitations. This prioritization helps to improve the efficiency and performance of the system while enhancing the quality of service and user experience of the vehicle. The Application Priority ensures that requesting vehicle is completed within the application completion deadline and that the requesting vehicle is not outside the communication range of the BS. The application priority is:

$$\bar{A}_m = \min \left(T_m^{\max}, \frac{\mathcal{L}_n^m}{v} \right) \quad (17)$$

$\bar{A}\bar{Q}$ is an application priority queue that stores multiple applications in ascending order of priority $\bar{A}_m$ in the queue.

Construct a task queue $\bar{T}\bar{Q}_m$ for multiple dependent tasks of the application m . The starting task is stored

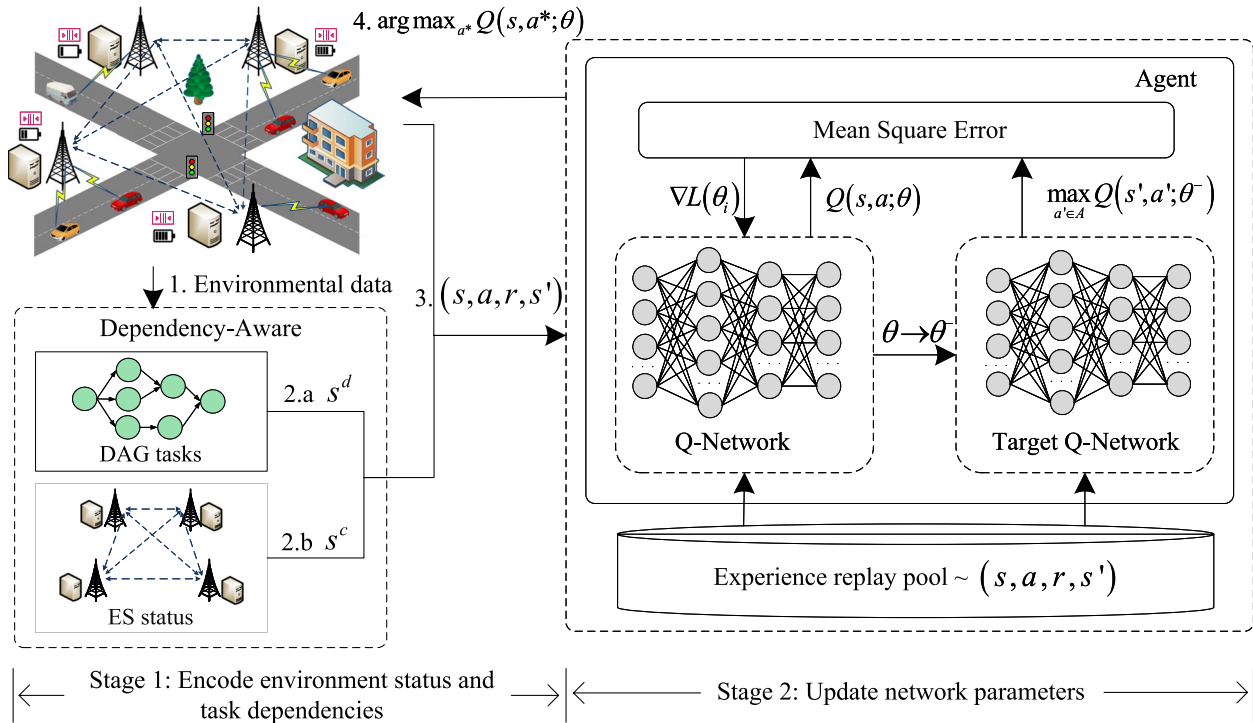


Fig. 3 DAG-DQN task dependency-aware offloading method

at the head of the queue, and the succeeding tasks are stored in $\bar{T}Q_m$ in descending order of $\bar{T}_{m,i}$. Repeat this process so that all tasks are stored in the task queue and the terminating tasks are stored at the end of the task queue. The task priority $\bar{T}_{m,i}$ is:

$$\bar{T}_{m,i} = \max_{j \in \text{sub}(i)} (\sigma \cdot c_i + (1 - \sigma) \cdot d_{i,j*}) \quad (18)$$

where σ denotes the trade-off factor $\sigma \in (0, 1)$.

Algorithm 1 DAG-DQN Method Offline Training

Require: ESs, APPs, $G, v, \mathcal{L}$;
Ensure: Task offloading decisions;

- 1: **for** int e=1 to Episode **do**
- 2: Initialize environment state s_t and neural network parameters θ ;
- 3: **for** int t=1 to T **do**
- 4: Use the ε -greedy policy to select the action a_t or the action with the largest potential payoff $a_t = \max_{a'} Q(s_t, a'; \theta^-)$ based on the current s_t ;
- 5: Perform the action a_t to get the reward r_t and the new state s_{t+1} according to Eq.(23);
- 6: Store (s_t, a_t, r_t, s_{t+1}) to the empirical replay pool D ;
- 7: Train the network with a random batch of samples from D ;
- 8: Construct the mean squared error function according to Eq.(25);
- 9: Update θ according to Eq.(26);
- 10: **if** s_{t+1} is Terminate **then**
- 11: End this round of iteration;
- 12: **return** $a_1, a_2, \dots, a_n$

DAG-DQN task dependency awareness

Applications and tasks are stored in queues according to their priority. Data dependencies between tasks are modeled as directed acyclic graphs that become part of the state of the environment perceived by the Agent. Agent makes different actions to interact with the environment state and the resulting state, action, reward and next state are stored in the experience replay pool. Redefine the reward function in the Q-network and iteratively train the Q-network through the Q-learning algorithm to get the optimal offloading strategy through multiple iterations. The DAG-DQN method is summarized in the Algorithm 1.

State Space: At moment t , the requesting vehicle uploads an application program to the ES within communication range, and the ES observes an environmental state with two components, a continuous data state s_t^c and a discrete data state s_t^d .

$$s_t^c = \{C_t, D_t, F_t, P_t', \mathcal{L}_k, v\} \quad (19)$$

$C_t = [c_1, c_2, \dots, c_n]$ denotes the set of computational data volume of all tasks under the application, and $D_t = [d_1, d_2, \dots, d_e]$ denotes the set of data volume transferred between tasks. $F_t = [f_1, f_2, \dots, f_K]$ and $P_t' = [P_1, P_2, \dots, P_K]$ denote the idle computational capacity and power consumption of all edge servers in the system, where $P_k = [P_k^{cal}, P_k^{trans}, P_k^{static}]$, $k \in K$. Some work predicts user movement trajectories and uses them as decision factors [34]. However, in this paper, the requesting vehicle can unload the task in a short time (within 1s), and the movement of the vehicle can be esti-

mated based on the position and instantaneous speed of the requesting vehicle. Therefore, the unloading decision in this paper does not consider the vehicle's trajectory. We take the vehicle's position information and velocity as inputs to the unloading decision.

It is worth noting that in neural networks, unlike continuous data, discrete data usually use an Embedding layer to convert discrete variables into continuous vectors, which are fed into the neural network together with other continuous data. By processing discrete data, the neural network pays more attention to the discrete data, to highlight the dependency between tasks and truly reflect the environment state before the task is unloaded. The discrete data state is:

$$s_t^d = \{X_t, G\} \quad (20)$$

Where $X_t = [x_1, x_2, \dots, x_K]$ denotes the set of all edge servers in the system environment, and the value of x_i is defined as the edge server number, $x_i = 1$ means that task i is offloaded to the edge server numbered 1.

X_t denotes the offloading scheme for all tasks under an application.

The dependencies between all tasks under an application are represented by a 0-1 encoded two-dimensional matrix G .

$$G = \begin{bmatrix} g_1^1 & g_1^2 & \dots & g_1^n \\ g_2^1 & g_2^2 & \dots & g_2^n \\ \vdots & \vdots & \ddots & \vdots \\ g_n^1 & g_n^2 & \dots & g_n^n \end{bmatrix} \quad (21)$$

Where $g_i^j \in \{0, 1\}$, when the j th column element of the i th row has a value of 1, it means that task j is the successor task of task i . When the element of the j th row of the i th column is 1, it means that task i is the predecessor task of task j . For example, $g_2^5 = 1$ indicates that task 5 is the successor task of task 2 and task 2 is the predecessor task of task 5. $g_2^5 = 0$ indicates that there is no dependency between task 5 and task 2.

Action Space: The action of the agent is to select the task to be offloaded to the ES under the current BS or transferred to the ES under another BS. When the agent receives the current environment state s_t , it quickly makes the task offloading action $a_t = [\Lambda_1, \Lambda_2, \dots, \Lambda_K]$, where $\Lambda_i \in \{0, 1\}$ and satisfies $\sum_{i=1}^K \Lambda_i = 1$, a_t denotes the offloading location of the task, and calculates the reward value R_t through the reward function. we adopt a single-step updating strategy, where the deep neural network returns a single task offloading action a_t , which represents the offloading scheme of a task, and the offloading scheme of all tasks in an application is $A_t = [a_1, a_2, \dots, a_n]$.

Reward Function: In reinforcement learning, the goal of the agent is to maximize the cumulative reward over time by learning to adjust its strategy. The cumulative reward is represented by the following equation:

$$R_t = \sum_{i=t}^T \gamma^{i-t} r_i \quad (22)$$

Equation (22) indicates that at moment t , the agent takes a series of actions from the current state to obtain the corresponding timely rewards $r_t, r_{t+1}, r_{t+2}, \dots, r_{T-1}, r_T$. They are weighted and summed to obtain the cumulative reward R_t . The discount factor γ represents the importance of future rewards which decreases over time. Future rewards will be more uncertain than current rewards and therefore need to be discounted.

The goal of DAG-DQN is to maximize the cumulative reward by training for the optimal task offloading method. If agent offloads task i to the ES at time t , it

can get the reward r_t in time. The normalized reward function is:

$$r_t = \lambda \left(\sum_{k \in \Psi(k)} \frac{1}{T_k^{cal}} + \sum_{p,q \in \Psi(k)} \frac{1}{T_{p,q}^{trans}} + \frac{1}{T_i^{cal}} \right) + \mu \left(\frac{1}{\frac{1}{N} \sum_{j=0}^N E_j} \right) \quad (23)$$

Where λ and μ denote the balancing factors, and $\Psi(k)$ denotes the set of all tasks preceding task k in the task queue $\overline{TQ}_m$. The reward obtained is inversely proportional to the task completion time and energy consumption. Reward value is larger when the task completion time and the energy consumption of the edge server are smaller.

Updating Network Parameters: In DAG-DQN method, parameter update of the deep neural network is mainly realized by using the mean square error loss function. A batch of random samples $S_t = \{(s_i, a_i, r_i, s') | i = 1, 2, \dots, H\}$ of size H is randomly drawn from the empirical replay pool. This batch of empirical samples is fed into the deep neural network. Each sample is computed with an estimated Q-value and a target Q-value, denoted as $Q(s_t, a_t; \theta)$ and y_t , respectively, with θ denoting the parameters of the deep neural network. The target Q-value is calculated by the equation:

$$y_t = \begin{cases} r_t, & \text{if } s_{t+1} \text{ is terminal} \\ r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-), & \text{otherwise} \end{cases} \quad (24)$$

Where θ^- denotes the parameter of the target network, which is fixed for a period of time and then periodically updated from θ^- to θ . The purpose of this is to make the target Q-value more stable and reduce the oscillation of the algorithm.

Next, loss value is calculated based on the mean square error loss function:

$$L(\theta) = \frac{1}{N} \sum_{t=1}^T (y_t - Q(s_t, a_t; \theta))^2 \quad (25)$$

Updating network parameters θ by gradient descent algorithm.

$$\theta_{i+1} = \theta_i - \alpha \nabla L(\theta_i) \quad (26)$$

Where α is the learning rate, which is used to control the step size of each update. $\nabla L(\theta_i)$ is the gradient vector of the loss function $L(\theta)$ at the parameter point θ_i , which represents the rate of change of the loss function for each parameter. Through iterative updating, the gradient descent algorithm can find the minimum value of the loss function.

Offline training of deep neural network models in DAG-DQN requires agents to constantly interact with the environment. The intelligent body learns the execution patterns and rules for task offloading from the environment state and quickly adjusts the parameters of the deep learning network. The trained model quickly formulates the task offloading method according to the environment state.

Experimental evaluation

This section is structured as follows, we first introduce the experimental simulation conditions. Secondly, we analyze the performance of DAG-DQN method in terms of application completion time, energy consumption, and completion rate through the experimental simulation results. Finally, we analyze the time complexity of the method and the execution time of the method.

Experimental settings

We use computational experiments to perform DAG-DQN task offloading experimental simulations. Communication methods, legal and institutional constraints, and real-world uncertainties in IoV systems are considered [35]. We use a truncated Gaussian distribution to generate vehicle mobility due to the traffic speed limitations required on urban highways [36]. The network environment in the experiment is established in an ideal situation, without considering abnormalities such as packet loss or interference.

We used Python simulator and Pycharm tools to write the simulation platform EdgeSim. Environment consists of 20 base stations, each equipped with an edge server, each with a bandwidth value randomly chosen from [10,50] Mbps and a processing capacity randomly chosen from $[0.5 \times 10^5, 10^5]$ KB per second. The coverage area of each base station is 1000 m. Each vehicle requests to offload an application, and each vehicle application consists of 7 tasks, the amount of data for the application is a randomly chosen value from the interval [100,300] KB, and the amount of data to be computed for each task is a randomly chosen value from [50,100] KB. The amount of data transferred between tasks is a randomly chosen value from the set [10,20] KB. We provide specific information about the tasks and data involved in the application during the simulation experiment. This is reflected in Table 3. The experimental parameters are summarized in Table 4.

Evaluation of the DAG-DQN method

To evaluate the effectiveness of the methods, DAG-DQN method was compared with several commonly used methods. The specific practices of the three methods are described below.

Table 3 Application data information.(Simulation experiment:2; Number of applications generated: 10; Application id:5.)

Task id	Calculated Data Size(KB)	Transmission Data Size(KB)
Task 0	88.88	29.21
Task 1	67.38	27.39
Task 2	50.25	38.50
Task 3	66.58	22.78
Task 4	54.0	27.92
Task 5	79.14	25.62
Task 6	93.48	0.0

Randomized Selection Method(RS): The RFF is a probability-based Method. This Method tends to find an approximate solution in the minimum time when dealing with NP-hard problems. By comparing RS with DAG-DQN, the efficiency and accuracy of DAG-DQN models in offloading decisions can be evaluated.

Optimal Greedy-based Method(OGOM) [37]: The OGOM is a local optimization Method. The decision that seems optimal at the moment is selected at each step without considering future effects. Therefore, it can obtain a near-optimal solution quickly. By comparing OGOM with DAG-DQN, the performance of DAG-DQN can be evaluated in terms of considering long-term effects and global optimization.

GA-based Sustainable Mobile Edge Server Selection(GAME)[38]: This is an improved genetic method. The authors used tournament selection to select the right number of solutions and cross mutation of chromosomes or solutions to get the best solution in the edge server selection problem. By comparing GAME to DAG-DQN, it is possible to assess the performance of DAG-DQN when dealing with complex problems and to understand its ability to adapt in the presence of a larger search space.

The specific experimental design is as follows. First, keep the number of edge servers constant and increase the number of applications. Observe the changes in the completion time of the total applications and the total energy consumption of the edge servers. Second, keep

Table 4 Parameter setting

Parameter	Meaning	Value
D_m^{up}	Uploading the application's data size(KB)	[100,300]
D_m^{down}	Vehicle receive data size(KB)	[50,100]
c_i	Task Calculation Data Size(KB)	[50,100]
$d_{i,j}$	Size of data transferred between tasks(KB)	[20,50]
H	Experience replay pool size	2000
α	Learning rate	0.001
γ	Discount factor	0.95

the number of applications constant and increase the number of edge servers. Observe the changes in the completion time of the total applications and the total energy consumption of the edge servers. The performance of DAG-DQN is evaluated in terms of reducing the total application completion time and total energy consumption compared to three methods, namely RS, OGOM, and GAME. In the simulation environment, two sets of experiments were conducted. In the first set of experiments, the number of applications was set to 5, 10, 15, and 20. In the second set of experiments, the number of edge servers was set to 10, 20, 30, and 40. Observe the impact of the number of applications and the number of edge servers on the task offloading decisions made by the four methods of generation.

The results of the first set of experimental comparisons are shown in Figs. 4, and 5. Increasing the number of applications leads to an increase in the amount of data that ESs need to compute, and therefore the total energy consumption and total application completion time gradually increase. Due to the limited computing resources of edge servers, multiple applications are queued for processing on a few edge servers. This increases the queuing latency of the applications and results in some edge servers being under high load for a long time. For the same

number of applications, the offloading decisions given by the DAG-DQN method have a lower total application completion time and total energy consumption than the offloading decisions given by the RS and OGOM methods. This demonstrates the superiority of the DAG-DQN method in reducing the optimization goals. However, it is slightly inferior to GAME. This is because GAME is a heuristic search method. This method can search for the global optimal solution in the entire search space thus obtaining better results. However the time complexity is high, convergence is slow and offloading decisions cannot be made in real time. In contrast, DAG-DQN can learn the optimal policy through online interaction with the environment with better convergence and stability, and can make uninstallation decisions in real time.

In the second set of experiments, the results of comparing the total application completion time and energy consumption of RS, GAME, OGOM, and DAG-DQN are shown in Figs. 6, and 7. An increase in the number of edge servers results in a larger number of edge servers that applications can choose to offload. Since the solution space expands as the number of edge servers increases, this may generate new offloading solutions. So the total application completion time decreases or remains the same. As the number of edge servers increases, the total

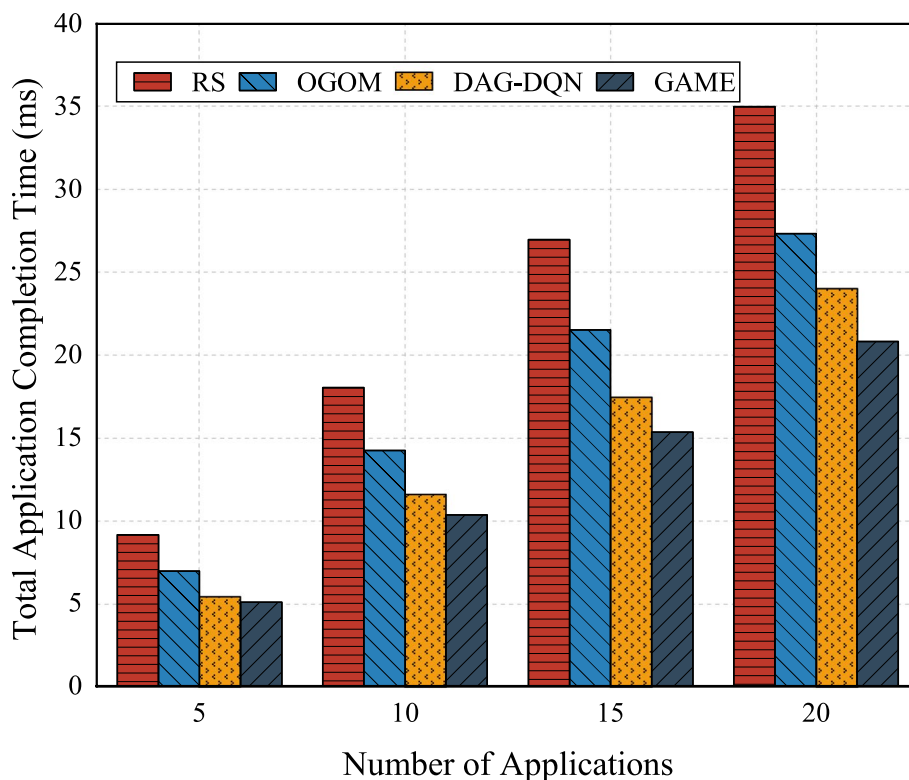


Fig. 4 Impact of the number of applications on application completion time

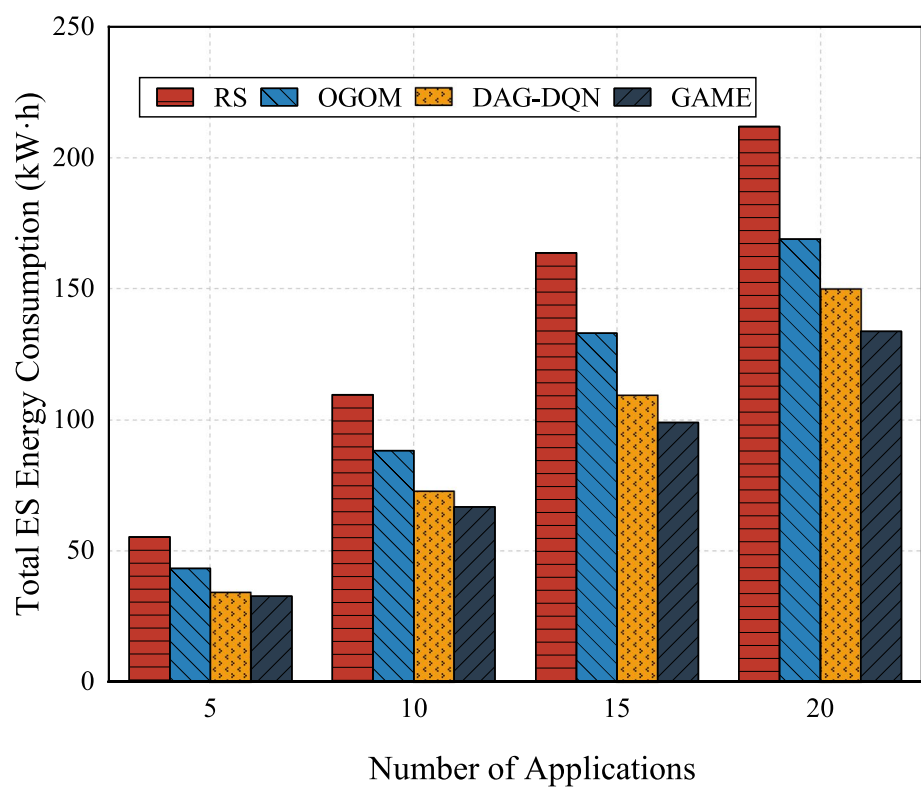


Fig. 5 Impact of the number of applications on the energy consumption of ESs

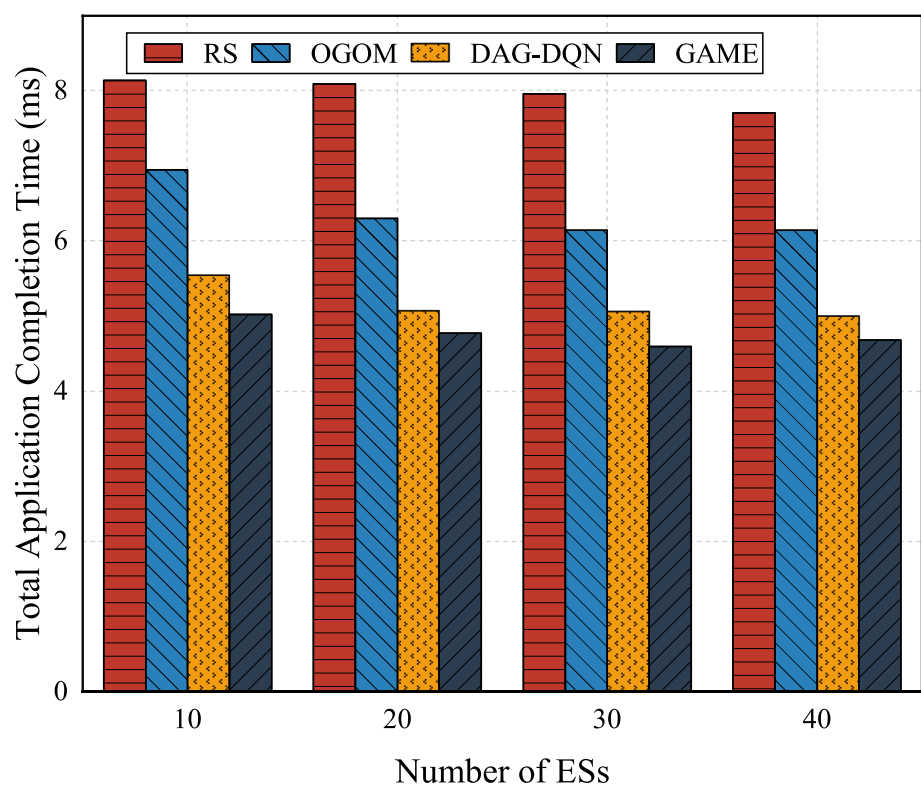


Fig. 6 Impact of the number of ESs on application completion time

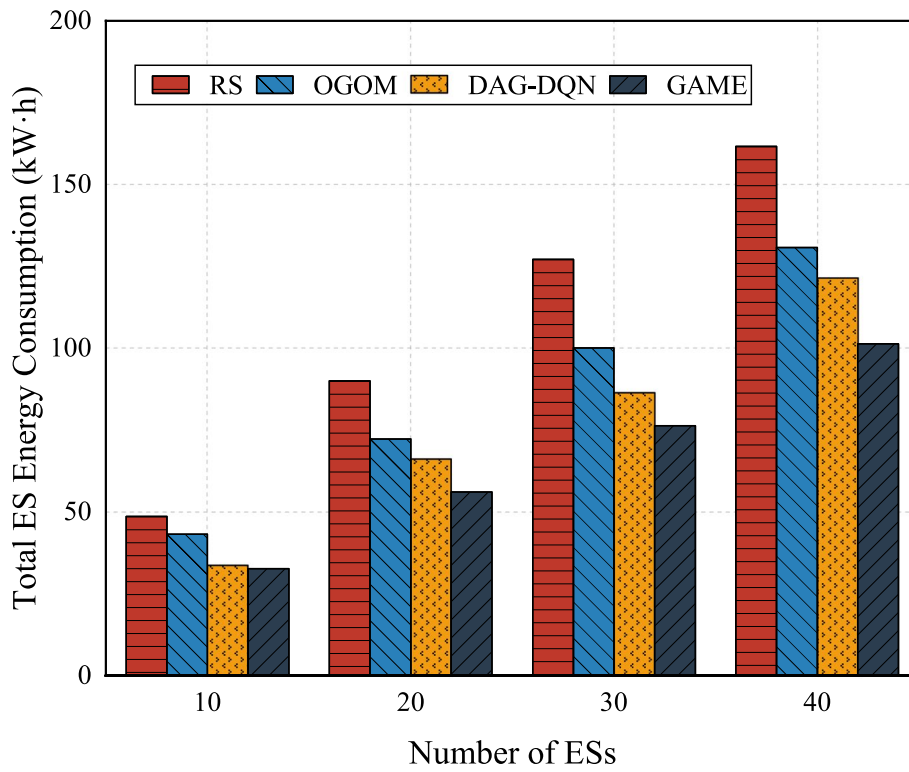


Fig. 7 Impact of the number of ESs on the energy consumption of ESs

energy consumption of edge servers also increases. With the same number of edge servers, DAG-DQN still outperforms RS and OGOM. It also shows that application completion times can be reduced by quickly deploying additional edge servers when task loads increase dramatically. In this paper, we focus on solving the task offloading problem using DAG-DQN, focusing on which edge servers the tasks are executed on. It does not consider how to quickly deploy additional edge servers.

Application completion rate

Equation (15) defines the completion rate of the application to evaluate whether the application can be processed within the time limit or before the vehicle moves out of communication range. The magnitude of the completion rate indicates whether the application can be successfully processed within the constraints.

The results of the completion rate comparison are shown in Figs. 8, and 9. In Fig. 8, as the number of applications increases, the application completion rate decreases for different methods. A requesting vehicle at the edge of the base station's communication range is present to unload the application. The requesting vehicle uploads the application to the base station and drives out of the communication range of the base station. This situation may result in the application not being completed

causing a decrease in the application completion rate. The advantages of GAME and DAG-DQN are demonstrated by comparing the completion rates of the same number of applications. In Fig. 9, the completion rate of the application gradually increases with the increase of edge servers. With the same number of edge servers, DAG-DQN and GAME have higher completion rates than RS and OGOM.

Method execution efficiency

The experiment was repeated several times while ensuring that the experimental conditions were the same, and the execution time required for the method to make the unloading decision was recorded and averaged each time. The experimental results are shown in Table 5.

n denotes the number of tasks of the application. m denotes the number of edge servers. p and g denote the population size and the number of iterations in the GAME method, respectively. As shown in Table 3, DAG-DQN spends much less execution time than GAME to formulate an offloading decision, with a four times shorter execution time. Although GAME works better in terms of solving capability, GAME has high time complexity and long decision time. GAME cannot fulfill the requirement of real-time application offloading. In autonomous driving, a delay of 1477ms may result in

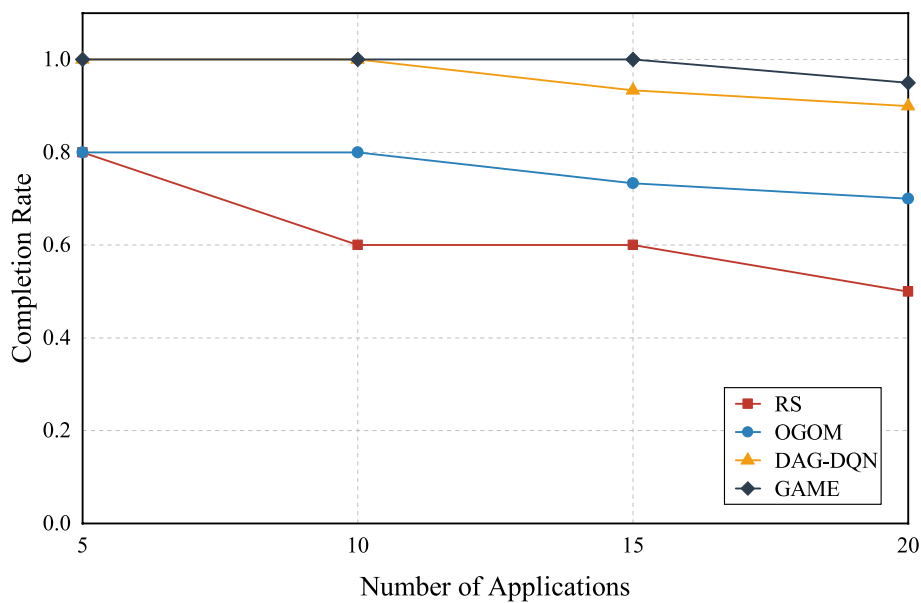


Fig. 8 Impact of the number of applications on the completion rate

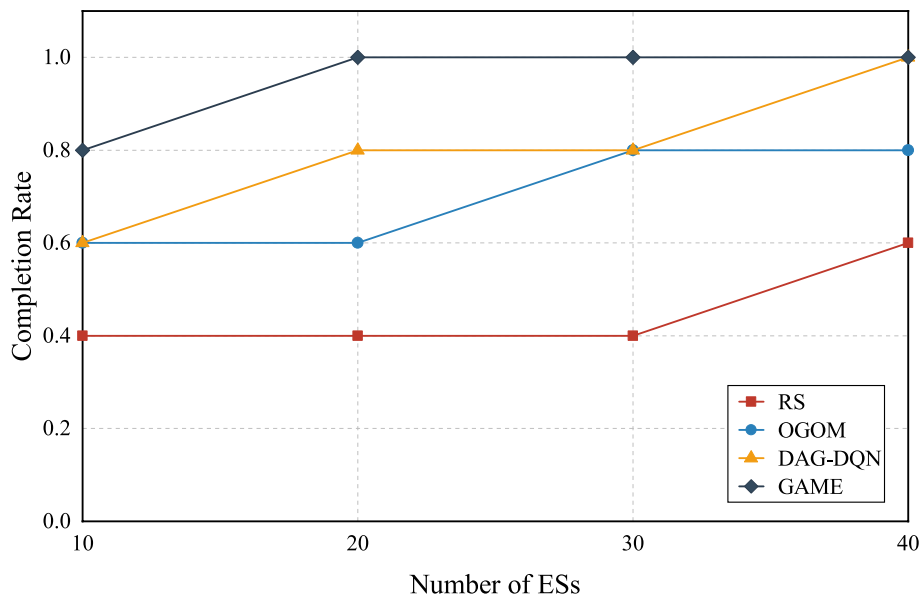


Fig. 9 Impact of the number of ESs on the completion rate

Table 5 Experimental result

Method name	Time complexity	Execution time (ms)
RS	$O(n)$	33
OGOM	$O(nm)$	123
GAME	$O(nmpg)$	1513
DAG-DQN	$O(n)$	36

the vehicle not being able to respond to changes in road conditions and other vehicles promptly, on time thus compromising the safety of the vehicle. In vehicle safety applications, a delay of 1477ms seconds may result in the system not being able to determine whether the driver is fatigued or distracted in time, thus affecting the safety and stability of the vehicle.

By analyzing the experimental results and method execution time, DAG-DQN can make offloading decisions efficiently and quickly, and meet the real-time application offloading requirements in IoV. The task offloading method should generate offloading decisions in a short time to meet the offloading requirements of computationally intensive and latency-sensitive vehicular applications. Also, task offloading methods should ensure that performance metrics such as latency and energy consumption are optimized. RS and OGOM have the same advantages as DAG-DQN in terms of execution time. However, they perform poorly in reducing application completion time and energy consumption. And RS and OGOM cannot guarantee the completion rate of vehicle applications. On the contrary, GAME excels in reducing application completion time and energy consumption with a high completion rate. However, the high complexity and long execution time of the method make it unsuitable for offloading decisions of real-time vehicle applications in IoV. It is concluded that DAG-DQN is more suitable for the task offloading in IoV. DAG-DQN has low execution time for fast decision making, which effectively reduces the application completion time and energy consumption and ensures the application completion rate.

Conclusion and future work

In this study, we propose a Dependency-aware online task offloading method for base station cooperative computing scenarios. It effectively reduces the total application completion time and total energy consumption and reduces the risk of application offloading failure and system overhead. We model applications as DAG tasks and define the priority of applications and tasks. ES stores applications and tasks in an execution queue based on priority. An ES using DAG-DQN method senses DAG tasks in the network environment and targets latency and energy reduction as the goal of task offloading decisions. Realizing real-time task offloading in IoV. Experimental results validate the superiority of the DAG-DQN task offloading method. The experimental results verified the superiority of the DAG-DQN method.

In the future, we aim to refine and bolster the performance of our task offloading methodology. This will entail the accumulation of extensive datasets pertaining to vehicular edge computing, enabling comprehensive validation of our approach across diverse real-world scenarios. In addition, we will improve our methodology to accommodate the dynamic and heterogeneous nature of vehicular environments, while ensuring seamless integration within existing IoV infrastructures. By addressing these key issues, we aspire to fortify the practical applicability and efficacy of our approach.

Abbreviations

IoV Internet of Vehicles

DAG	Directed Acyclic Graph
DAG-DQN	Based on task dependency awareness in deep reinforcement learning
BS	Base Station
ES	Edge Server
MDP	Markov Decision Process
MIMO	Multiple Input Multiple Output
RS	Randomized Selection Method
OGOM	Optimal Greedy-based Method
GAME	GA-based Sustainable Mobile Edge Server Selection

Acknowledgements

Not applicable.

Authors' contributions

Chunhong Liu provided methodological ideas and experimental design. Huai-chen Wang completed the experiment and wrote the paper. Mengdi Zhao assisted in data acquisition, processing and analysis. Xiaoyan Zhao assisted system model. Jialei Liu and Peiyan Yuan proposed changes to the paper as a whole. All authors reviewed the manuscript.

Funding

This work was supported in part by the National Natural Science Foundation of China (No.62272031,62072159), the Natural Science Fund Project of Hubei Province (No.2023AFB585), the Open Project of the State Key Laboratory of Networking and Switching Technology (BUPT) under Grant SKLNT-2020-1-02, the Applied Research Program of Key Research Projects of Henan Higher Education Institutions (No.23A520036) and the Major Project of Hubei Province Science and Technology (No.2023BCA006), the Hubei Superior and Distinctive Discipline Group of "New Energy Vehicle and Smart Transportation".

Availability of data and materials

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

The coauthors all agree on the paper publication.

Competing interests

The authors declare no competing interests.

Received: 1 April 2024 Accepted: 30 August 2024

Published online: 05 September 2024

References

- Hamdi W, Ksouri C, Bulut H et al (2024) Network Slicing Based Learning Techniques for IoV in 5G and Beyond Networks[J]. *IEEE Commun Surv Tutor*. <https://doi.org/10.1109/COMST.2024.337208>
- Attaran M (2023) The impact of 5G on the evolution of intelligent automation and industry digitization[J]. *J Ambient Intell Humanized Comput* 14(5):5977–5993
- Wang F, Zhu H, Srivastava G et al (2021) Robust collaborative filtering recommendation with user-item-trust records[J]. *IEEE Trans Comput Soc Syst* 9(4):986–996
- Liu Y, Zhou X, Kou H et al (2023) Privacy-preserving point-of-interest recommendation based on simplified graph convolutional network for geological traveling[J]. *ACM Trans Intell Syst Technol*. <https://doi.org/10.1145/3620677>
- Liu Y, Zhang Y, Mao X et al (2024) Lithological Facies Classification Using Attention-Based Gated Recurrent Unit[J]. *Tsinghua Sci Technol* 29(4):1206–1218

6. Qi L, Liu Y, Zhang Y et al (2022) Privacy-aware point-of-interest category recommendation in internet of things[J]. *IEEE Internet Things J* 9(21):21398–21408
7. Zhou H, Xu W, Chen J et al (2020) Evolutionary V2X technologies toward the Internet of vehicles: Challenges and opportunities[J]. *Proc IEEE* 108(2):308–323
8. Liu C, Zhao M, Wang H et al (2024) Stackelberg-Game Computation Offloading Scheme for Parked Vehicle-Assisted VEC and Experiment Analysis[J]. *IEEE Trans Intell Veh.* <https://doi.org/10.1109/TIV.2024.3357076>
9. Liu J, Li G, Huang Q et al (2023) Cooperative resource allocation for computation-intensive IIoT applications in aerial computing[J]. *IEEE Internet Things J* 10(11):9295–9307
10. Bodkhe U, Tanwar S (2022) P 2 COMM: A Secure and Low-Cost Message Dissemination Scheme for Connected Vehicles[C]. In: *IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, p 1–6. <https://doi.org/10.1109/INFOCOMWKSHPS54753.2022.9798285>
11. Qi L, Xu X, Wu X et al (2023) Digital-twin-enabled 6g mobile network video streaming using mobile crowdsourcing[J]. *IEEE J Sel Areas Commun* 41(10):3161–3174
12. Meng K, Liu Z, Xu X et al (2023) Heterogeneous Edge Service Deployment for Cyber Physical Social Intelligence in Internet of Vehicles[J]. *IEEE Trans Intell Veh.* <https://doi.org/10.1109/TIV.2023.3325372>
13. Dai Y, Xu D, Maharjan S et al (2018) Joint load balancing and offloading in vehicular edge computing and networks[J]. *IEEE Internet Things J* 6(3):4377–4387
14. Zhang T, Zhou X, Liu J et al (2023) QoE-Driven Data Communication Framework for Consumer Electronics in Tele-Healthcare System[J]. *IEEE Trans Consum Electron* 69(4):719–733
15. Huang J, Ma B, Wang M et al (2024) Incentive mechanism design of federated learning for recommendation systems in MEC[J]. *IEEE Trans Consum Electron* 70(1):2596–2607
16. Liu L, Chen C, Pei Q et al (2021) Vehicular edge computing and networking: A survey[J]. *Mob Netw Appl* 26:1145–1168
17. Li Z, Wang Q, Chen YF et al (2021) A survey on task offloading research in vehicular edge computing[J]. *Chin J Comput* 44(5):963–982
18. Guan F, Peng L, Qiao J (2023) A new federated scheduling algorithm for arbitrary-deadline DAG tasks[J]. *IEEE Trans Comput.* <https://doi.org/10.1109/TC.2023.3244632>
19. Fan W, Liu J, Hua M et al (2022) Joint task offloading and resource allocation for multi-access edge computing assisted by parked and moving vehicles[J]. *IEEE Trans Veh Technol* 71(5):5314–5330
20. Zhou C, Wu W, He H et al (2020) Deep reinforcement learning for delay-oriented IoT task scheduling in SAGIN[J]. *IEEE Trans Wirel Commun* 20(2):911–925
21. Deng X, Yin J, Guan P et al (2021) Intelligent delay-aware partial computing task offloading for multiuser industrial internet of things through edge computing[J]. *IEEE Internet Things J* 10(4):2954–2966
22. Shinde SS, Tarchi D (2023) A Markov Decision Process Solution for Energy-Saving Network Selection and Computation Offloading in Vehicular Networks[J]. *IEEE Trans Veh Technol.* <https://doi.org/10.1109/TVT.2023.3264504>
23. Ning Z, Huang J, Wang X et al (2019) Mobile edge computing-enabled internet of vehicles: Toward energy-efficient scheduling[J]. *IEEE Netw* 33(5):198–205
24. Maleki H, Basaran M, Durak-Ata L (2023) Handover-Enabled Dynamic Computation Offloading for Vehicular Edge Computing Networks[J]. *IEEE Trans Veh Technol.* <https://doi.org/10.1109/TVT.2023.3247889>
25. Zhao H, Zhang T, Chen Y et al (2020) Task distribution offloading algorithm of vehicle edge network based on DQN[J]. *J Commun* 41(10):172–178
26. Liu Y, Wang S, Zhao Q et al (2020) Dependency-aware task scheduling in vehicular edge computing[J]. *IEEE Internet Things J* 7(6):4961–4971
27. Qi Q, Wang J, Ma Z et al (2019) Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach[J]. *IEEE Trans Veh Technol* 68(5):4192–4203
28. He K, Meng X, Pan Z et al (2018) A novel task-duplication based clustering algorithm for heterogeneous computing environments[J]. *IEEE Trans Parallel Distrib Syst* 30(1):2–14
29. Liu Z, Xu X, Han F et al (2023) Secure Edge Server Placement with Non-Cooperative Game for Internet of Vehicles in Web 3.0[J]. *IEEE Trans Netw Sci Eng.* <https://doi.org/10.1109/TNSE.2023.3321139>
30. Liu W, Xu X, Qi L et al (2024) Digital Twin-Assisted Edge Service Caching for Consumer Electronics Manufacturing[J]. *IEEE Trans Consum Electron.* <https://doi.org/10.1109/TCE.2024.3357136>
31. Niu M, Cheng B, Feng Y et al (2020) GMTA: A geo-aware multi-agent task allocation approach for scientific workflows in container-based cloud[J]. *IEEE Trans Netw Serv Manag* 17(3):1568–1581
32. Xu X, Yang C, Bilal M et al (2022) Computation offloading for energy and delay trade-offs with traffic flow prediction in edge computing-enabled iov[J]. *IEEE Trans Intell Transp Syst.* <https://doi.org/10.1109/TITS.2022.3221975>
33. Loyka SL (2001) Channel capacity of MIMO architecture using the exponential correlation matrix[J]. *IEEE Commun Lett* 5(9):369–371
34. Zaman SK, Jehangiri AI, Maqsood T et al (2023) LiMPO: Lightweight mobility prediction and offloading framework using machine learning for mobile edge computing[J]. *Cluster Comput* 26(1):99–117
35. Xue X, Yu X, Zhou D et al (2023) Computational experiments for complex social systems-Part III: the docking of domain models[J]. *IEEE Trans Comput Soc Syst.* <https://doi.org/10.1109/TCSS.2023.3243894>
36. Yu Z, Hu J, Min G et al (2020) Mobility-aware proactive edge caching for connected vehicles using federated learning[J]. *IEEE Trans Intell Transp Syst* 22(8):5341–5351
37. Zhou W, Lin C, Duan J et al (2021) An optimized greedy-based task offloading method for mobile edge computing. In: *International Conference on Algorithms and Architectures for Parallel Processing*. Springer International Publishing, Cham, pp 494–508
38. Chakraborty S, Mazumdar K (2022) Sustainable task offloading decision using genetic algorithm in sensor mobile edge computing[J]. *J King Saud Univ Comput Inform Sci* 34(4):1552–1568

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.